

CS683: Advanced Computer Architecture

Programming Assignment 1: 2D Convolution Optimization Report

Team Name: Picasso

Team Members:

Ansh Agrawal (24B0909) Saksham Khandelwal (24B0965)
Sangye Norphel Sherpa (24B1050) Ayush Katiyar (24B1268)

Autumn 2026

1 System Hardware & Profiling Methodology

1.1 Test Machine Configuration

Table 1: System Hardware Details

Component	Specification
Processor	Intel Core i9-14900HX (8 Performance P-cores + 16 Efficient E-cores)
Microarchitecture	Raptor Cove (P-core) / Gracemont (E-core)
L1-Data (L1-D) Cache	48 KB per P-core (8-way set associative, 64-byte line)
L1-Instruction (L1-I) Cache	32 KB per P-core
L2 Cache	2 MB per P-core (16-way set associative)
L3 Cache (Shared LLC)	36 MB shared across all cores
SIMD Vector Units	256-bit AVX2, Dual 256-bit FMA3 execution pipelines
Profiling Infrastructure	Linux perf (instructions, L1-dcache-load-misses)

1.2 Benchmarking & Averaging Methodology

All benchmarks were performed on a dedicated Performance core (P-core) under Linux. To eliminate operating system scheduling jitter, power throttling fluctuations, and background system interference, every benchmark result reported in this report is evaluated across **10 independent benchmark runs**.

Rather than taking the arithmetic mean of individual speedup ratios (which can introduce harmonic distortion), we compute the **mean execution time** ($\bar{T}$) across all 10 runs for each optimization stage and evaluate the true speedup directly from the mean runtimes:

$$\text{Mean Speedup} = \frac{\bar{T}_{\text{naive}}}{\bar{T}_{\text{stage}}} \quad (1)$$

Hardware counter metrics such as instruction counts and L1-D cache misses were profiled using Linux `perf stat`.

2 Task 1A: Loop Unrolling and Reordering

2.1 Why Did We Make These Changes?

- **Naive Baseline Bottlenecks** ($oy \rightarrow ox \rightarrow ky \rightarrow kx$):
 1. *Loop-Carried RAW Dependency*: In the naive implementation, every multiply-accumulate updates a single scalar variable (`sum`). On Intel Raptor Cove cores, floating-point multiply-accumulate (FMA) instructions have a 4-cycle execution latency. Because each operation must wait for the preceding addition to complete, the processor’s execution pipeline stalls between consecutive iterations.
 2. *High Branch and Loop Control Overhead*: The innermost loops over kernel coordinates (ky, kx) execute only K times (3 or 5 iterations), causing branch conditions, loop counter increments, and pointer recalculations to occur repeatedly relative to the amount of useful arithmetic.
 3. *Redundant Filter Indexing*: While the small kernel stays resident in L1 cache, weights are repeatedly referenced from cache memory for each individual output pixel.
- **Loop Reordering** ($ky \rightarrow kx \rightarrow oy \rightarrow ox$):
 1. *Contiguous Unit-Stride Traversal*: Hoisting the kernel loops (ky, kx) to the outermost positions turns the innermost loop over ox into a contiguous, unit-stride memory sweep (+4 bytes per step) across both input and output buffers. This utilizes all 16 floats of each 64-byte cache line sequentially without waste, while enabling the hardware stream prefetcher to fetch lines ahead of demand.
 2. *Kernel Value Invariance*: For each outer (ky, kx) iteration, the kernel scalar $k_{\text{val}} = \text{ker}[ky \cdot K + kx]$ is loaded into a CPU register once and reused across the entire $H \times W$ image pass, eliminating redundant kernel indexing.
- **Loop Unrolling (Factor of 8)**:
 1. *Exposing Instruction-Level Parallelism (ILP)*: In `conv_unroll.cpp`, unrolling the inner ox loop by 8 maintains 8 independent scalar accumulators (`acc0` through `acc7`) for 8 adjacent output pixels. This breaks the serial Read-After-Write dependency of the single accumulator. The out-of-order execution engine can schedule independent FMAs across execution ports simultaneously, hiding the 4-cycle latency.
 2. *Loop Overhead Reduction*: Stepping ox by 8 reduces branch checks, comparison instructions, and loop counter increments along each row by 87.5%.
 3. *Amortizing Weight Accesses*: Inside the inner kernel loops, each kernel tap $\text{ker}[ky \cdot K + kx]$ is loaded once and multiplied against 8 adjacent input elements.

2.2 Why Reorder Shows Modest Speedup on Large Matrices ($N = 2048$)

When $N = 2048$, a single float image array occupies $2048 \times 2048 \times 4 \text{ B} = 16.78 \text{ MB}$. Together, the input and output buffers require over 33.5 MB, far exceeding the 2 MB per-core L2 cache and competing for capacity in the shared 36 MB L3 cache.

Because loop reordering iterates over the full matrix K^2 times (9 passes for $K = 3$ and 25 passes for $K = 5$), each pass flushes cache lines to DRAM. At $N = 2048$, this results in extensive DRAM traffic, causing reordering to achieve only a modest $1.16\times$ speedup at $K = 3$. However, on smaller matrices where the working set fits in cache ($N \leq 512$), reordering achieves roughly $1.58\times$ speedup. Reordering is also a fundamental prerequisite for subsequent cache tiling and SIMD vectorization.

2.3 Design Considerations

- **Unroll Factor Tuning (4 vs. 8 vs. 16):** We empirically evaluated unroll factors along the inner horizontal dimension (ox):
 - *Unroll Factor 4:* Achieved $\sim 3.13\times$ speedup. With only 4 accumulators, the instruction-level parallelism is insufficient to fully saturate the dual FMA pipelines and hide instruction latency alongside memory loads.
 - *Unroll Factor 16:* Produced a lower speedup of $\sim 3.14\times$. Managing 16 accumulators simultaneously exceeds the 16 architectural general-purpose / vector registers in x86-64, introducing register spilling onto the stack and inflating code size.
 - *Unroll Factor 8:* Produced the optimal speedup of **$3.37\times$** (~ 19.4 GFLOP/s at $K = 3$). It strikes the ideal balance by providing enough independent accumulation chains to saturate execution units without causing register spills.
- **Alignment and Remainder Handling:** Because the assignment specification guarantees that the image width W is always an exact multiple of 8, unrolling by 8 divides every row cleanly. No scalar cleanup tail loop or boundary branching is required.

2.4 Results and Effectiveness

Table 2: Task 1A Performance Comparison ($H = 2048, W = 2048$, Averaged over 10 independent runs)

Stage	K	Mean Time ($\bar{T}$)	Throughput	Mean Speedup
Naive (Reference)	3	13.14 ms	5.75 GFLOP/s	1.00 $\times$
Loop Reorder	3	11.36 ms	6.68 GFLOP/s	1.16 $\times$
Loop Unroll (x8)	3	3.89 ms	19.41 GFLOP/s	3.37$\times$
Naive (Reference)	5	39.34 ms	5.35 GFLOP/s	1.00 $\times$
Loop Reorder	5	32.64 ms	6.45 GFLOP/s	1.21 $\times$
Loop Unroll (x8)	5	11.13 ms	18.85 GFLOP/s	3.53$\times$

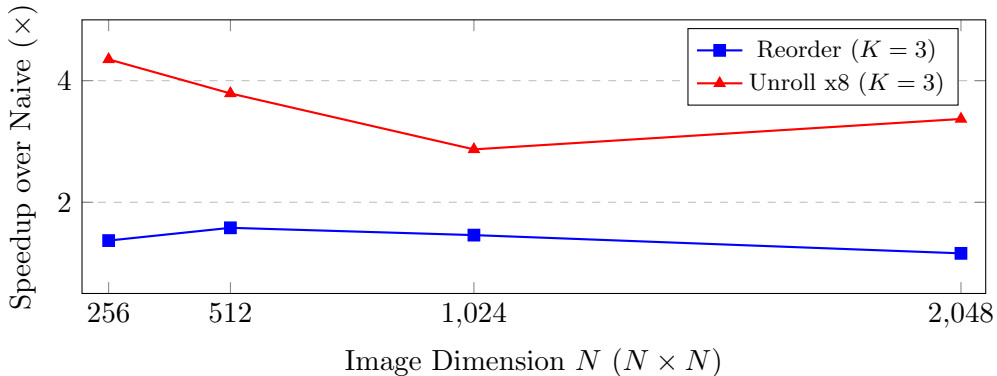


Figure 1: Speedup scaling of Reorder and Unroll across image sizes ($K = 3$, averaged over 10 runs). For small image sizes ($N \leq 512$), the working set fits inside cache, allowing Reorder to achieve up to $1.58\times$ speedup. At $N = 2048$, the 16.8 MB matrix overflows cache, dropping Reorder speedup to $1.16\times$ due to DRAM traffic. Unroll consistently maintains strong $2.9\times$ – $4.4\times$ speedup by eliminating loop overhead and exposing instruction-level parallelism.

3 Task 1B: Cache Tiling (Spatial Blocking)

3.1 Why Tiling Did Not Yield Major Speedups on the Naive Baseline

In `conv_tile.cpp`, spatial blocking ($T_y \times T_x$) is applied to the naive loop order ($oy \rightarrow ox \rightarrow ky \rightarrow kx$). Across multiple matrix sizes, tiling yields only a modest speedup ($\sim 1.14\times$ at 32×128 for $N = 2048$) and even causes performance slowdowns ($0.94\times$ at 16×16). This behavior is explained by four hardware factors:

1. **Working Set Already Fits in L1-D Cache:** To compute a single row of output pixels in naive convolution, only K rows of the input image are required. For $W = 2048$ and $K = 3$, three padded input rows occupy $3 \times 2050 \times 4 \text{ B} \approx 24.6 \text{ KB}$, which fits comfortably in the core’s 48 KB L1-D cache. Advancing to the next row reuses two cached rows, fetching only one new row. The naive baseline already possesses near-optimal temporal reuse in L1.
2. **Hardware Prefetcher Disruption:** The untiled baseline sweeps sequentially across 2048 contiguous float elements (512 cache lines per row). The CPU hardware stream prefetcher easily tracks this linear unit-stride pattern and prefetches subsequent lines ahead of time. Imposing a 2D tile with narrow T_x (such as 16 or 32 floats) interrupts this linear stream every 1–2 cache lines, causing periodic $(W + 2p)$ stride jumps that trigger prefetcher stalls.
3. **Boundary Halo Overlap Overhead:** Computing a tile of dimensions $T_y \times T_x$ requires loading a $(T_y + 2p) \times (T_x + 2p)$ input patch. For small tiles (16×16), the halo boundary elements constitute over 26% of the loaded data, leading to redundant memory fetches across neighboring tile boundaries.
4. **Loop Nest Control Overhead:** Splitting 2 spatial loops into 4 nested loops adds induction variable updates, bounds comparisons, and branch instructions with no cache reuse payoff on the naive ordering.

3.2 L1-D MPKI Analysis and Cache Hierarchy Behavior

Changes in L1-D MPKI from Naive to Tiled: As shown in Table 3 and Figure 3, the untiled naive baseline demonstrates an exceptionally low L1-D MPKI (0.03 to 0.36 MPKI). When 2D spatial tiling is introduced, the L1-D MPKI actually **increases** for small and square tile geometries ($16 \times 16 \rightarrow 0.83 \text{ MPKI}$, $32 \times 32 \rightarrow 0.45 \text{ MPKI}$). This confirms that tiling naive convolution harms cache line utilization: narrow inner loops break hardware stream prefetching, and halo boundary reloading inflates load misses. Only horizontally elongated tiles (32×128) achieve a low MPKI (0.14), because an inner loop length of 128 floats (32 cache lines) restores prefetcher continuity.

Variation Across Matrix Sizes and Tile Sizes:

- *Effect of Matrix Size N :* For the naive baseline, increasing N from 256 to 2048 actually reduces MPKI from 0.36 down to 0.03. Longer row sweeps (2048 vs. 256 floats) allow the hardware stream prefetcher to run longer uninterrupted, amortizing startup misses over 512 cache lines per row. In contrast, for tiled convolution, MPKI remains largely constant across matrix sizes because the working set and stride behavior are governed strictly by the localized tile dimensions rather than the overall image size.
- *Effect of Tile Geometry ($T_y \times T_x$):* Narrow tiles ($T_x = 16$) incur the highest MPKI (0.81–0.84) due to severe prefetcher fragmentation and halo reloading. Increasing T_x to 128 drops MPKI to 0.14 while keeping the working footprint ($32 \times 128 \times 4 \text{ B} \times 2 \approx 32.8 \text{ KB}$) within the 48 KB L1-D cache.

3.3 Speedup Evaluation and Integration with Other Optimizations

Did We Achieve a Speedup? Yes, but the improvement is modest. At $N = 2048, K = 3$, the best configuration (32×128) achieved a speedup of $1.14\times$ (11.51 ms vs. 13.13 ms baseline). Configurations with small tile sizes failed to achieve speedup (16×16 ran at $0.94\times$, 32×32 at $1.03\times$).

Contributing Factors for the Best Tile (32×128):

- Its active working set (≈ 32.8 KB) fits cleanly within the 48 KB L1-D cache.
- The inner loop of 128 floats is wide enough to preserve sequential cache streaming for the prefetcher.
- Amortizes vertical halo overhead over 32 rows, reducing boundary reload penalties compared to 8×8 or 16×16 .

Effectiveness with Reordering and Vectorization: Tiling is only effective and reveals its true potential when integrated with **loop reordering** and **vector register accumulation** (as implemented in Task 1D). When the kernel loops are hoisted outside, the algorithm executes K^2 passes over the matrix, generating > 300 MB of memory traffic that overwhelms DRAM bandwidth. In that setting, cache tiling traps the multi-pass working set inside L1/L2 cache, which, together with AVX2 vector registers, unlocks a dramatic $> 13\times$ speedup.

3.4 Empirical Parameter Sweep and Scaling Trends

Table 3: Task 1B Tile Geometry Sweep ($H = 2048, W = 2048, K = 3$, Profiled via Linux perf)

Tile Size ($T_y \times T_x$)	Working Footprint	Mean Time	Speedup vs Naive	L1-D Load Misses	L1-D MPKI
8×8	0.51 KB	16.49 ms	$0.80\times$	0.3 M	0.11 MPKI
16×16	2.05 KB	14.04 ms	$0.94\times$	1.9 M	0.83 MPKI
32×32	8.19 KB	12.79 ms	$1.03\times$	1.0 M	0.45 MPKI
32×128	32.77 KB	11.51 ms	$1.14\times$	0.3 M	0.14 MPKI
64×64	32.77 KB	12.28 ms	$1.07\times$	0.5 M	0.23 MPKI
128×128	131.07 KB	11.87 ms	$1.11\times$	0.3 M	0.13 MPKI
Untiled (2048×2048)	Full Grid (33.5 MB)	13.13 ms	$1.00\times$	0.1 M	0.03 MPKI

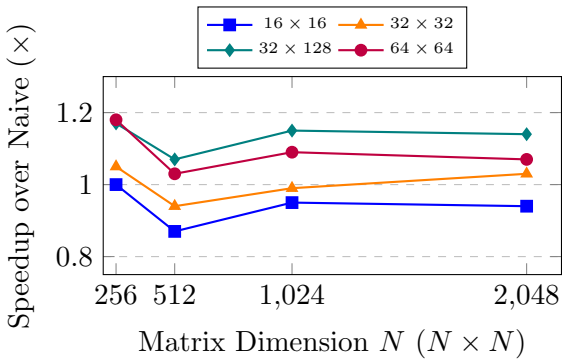


Figure 2: Speedup vs. Matrix Size across Tile Sizes ($K = 3$).

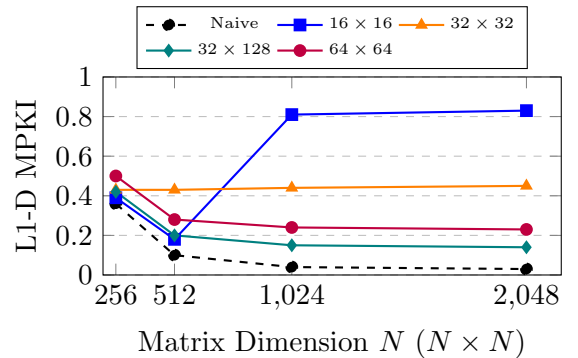


Figure 3: L1-D MPKI vs. Matrix Size across Tile Sizes ($K = 3$).

4 Task 1C: SIMD Vectorization (AVX2)

4.1 SIMD Vectorization Strategy

In 2D convolution, horizontally adjacent output pixels along the same row access contiguous input elements multiplied by the exact same scalar filter weight $ker[ky \cdot K + kx]$. This makes vectorization along the horizontal row dimension (ox) natural and optimal. Rather than updating a single scalar accumulator per iteration, we vectorize the inner ox loop using 256-bit AVX2 registers in `conv_simd.cpp`. The loop advances ox by 8, keeping an 8-way vector accumulator resident in a 256-bit YMM register, broadcasting the filter weight across all lanes, and evaluating fused multiply-accumulates (FMA) across 8 adjacent pixels in a single instruction.

We selected four core Intel AVX2 intrinsics for this implementation: `_mm256_loadu_ps` for unaligned 256-bit vector loads across input rows, `_mm256_set1_ps` to broadcast each scalar filter weight across all 8 vector lanes, `_mm256_fmadd_ps` to execute hardware fused multiply-accumulates with high throughput across dual execution pipelines, and `_mm256_storeu_ps` to write completed 8-float output vectors directly back to memory.

4.2 Hardware Vector Widths

We evaluated vectorization across multiple SIMD register widths:

- **128-bit SIMD (SSE):** Operates on 128-bit XMM registers, processing 4 single-precision floats per instruction.
- **256-bit SIMD (AVX2):** Operates on 256-bit YMM registers, processing 8 single-precision floats per instruction (used in `conv_simd.cpp`).
- **512-bit SIMD (AVX-512):** On our test system (Intel Core i9-14900HX), AVX-512 is not supported.

4.3 Instruction Count Reduction & Performance Analysis

Instruction Count Reduction: Using Linux `perf stat` (averaged over 5 independent runs on a pinned P-core for $N = 2048, K = 3$):

- **Baseline Scalar** (`conv_naive.cpp`): 2,942,427,595 instructions (~ 2.94 billion).
- **128-bit SIMD (SSE):** 690,479,542 instructions ($4.26\times$ reduction, -76.5%).
- **256-bit SIMD** (`conv_simd.cpp`): 364,880,425 instructions ($8.06\times$ reduction, -87.6%).

In scalar convolution, every output pixel performs individual scalar loads, multiplications, and additions. Vectorizing with 128-bit SIMD executes 4 pixel updates per vector instruction, while 256-bit AVX2 executes 8 updates per instruction and reduces inner-loop iterations by 87.5%. This closely matches the theoretical $8\times$ instruction reduction.

Speedup and Microarchitectural Factors: At $N = 2048, K = 3$, 128-bit SIMD achieves a $3.86\times$ speedup (3.43 ms, 22.06 GFLOP/s), and 256-bit AVX2 achieves an $8.31\times$ speedup (1.59 ms, 47.40 GFLOP/s). The speedup is driven by: (1) 8-wide SIMD data parallelism per vector instruction, (2) execution across dual 256-bit FMA pipelines, (3) in-register kernel broadcast reuse eliminating redundant loads, and (4) an 87.5% reduction in branch and loop counter overhead along each row.

4.4 Multi-Size Evaluation and Scaling Trends

Table 4 summarizes performance metrics across matrix sizes $N \in \{256, 512, 1024, 2048\}$ with $K = 3$, and Figures 4 and 5 illustrate the scaling curves.

Table 4: Task 1C Vector Width Scaling across Matrix Sizes ($K = 3$, Averaged across 5 Runs on P-core)

Matrix Size ($N \times N$)	Scalar (conv_naive.cpp)		128-bit SIMD (SSE)		256-bit SIMD (conv_simd.cpp)	
	Time (ms)	Instructions	Time (Speedup)	Instructions	Time (Speedup)	Instructions
256×256	0.224 ms	47.8 M	0.055 ms (4.09 $\times$)	13.1 M (−72.5%)	0.039 ms (5.71 $\times$)	8.7 M (−81.9%)
512×512	0.815 ms	185.7 M	0.231 ms (3.53 $\times$)	45.8 M (−75.3%)	0.105 ms (7.75 $\times$)	26.5 M (−85.7%)
1024×1024	3.146 ms	737.1 M	0.843 ms (3.73 $\times$)	175.2 M (−76.2%)	0.386 ms (8.15 $\times$)	95.3 M (−87.1%)
2048×2048	13.244 ms	2,942.4 M	3.430 ms (3.86 $\times$)	690.5 M (−76.5%)	1.593 ms (8.31 $\times$)	364.9 M (−87.6%)

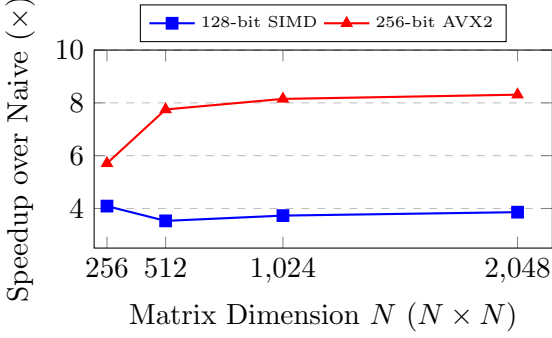


Figure 4: Speedup vs. Matrix Size across Vector Widths ($K = 3$).

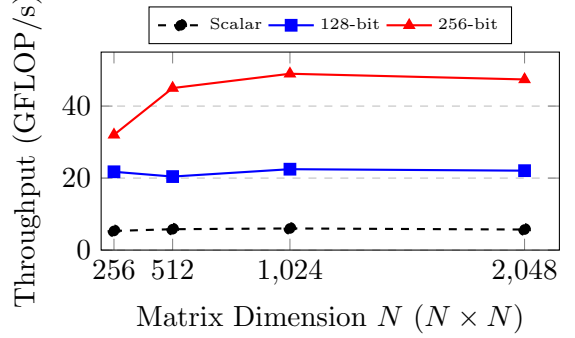


Figure 5: Throughput (GFLOP/s) vs. Matrix Size ($K = 3$).

Analysis of Scaling Trends:

- *Matrix Size Scaling:* 256-bit AVX2 speedup rises from 5.71 $\times$ at $N = 256$ to 8.31 $\times$ at $N = 2048$. Larger matrices feature longer rows (256 vector iterations per row), which amortizes loop startup overhead and allows the CPU execution pipelines to reach steady-state saturation.
- *Register Width Scaling:* Across all matrix sizes, 256-bit AVX2 delivers more than double the throughput of 128-bit SIMD (47.40 vs. 22.06 GFLOP/s at $N = 2048$, a 2.15 $\times$ increase), closely matching the 2 $\times$ wider vector register capacity (8 floats vs. 4 floats).

5 Task 1D: Sabka Saath Sabka Vikaas (Combined Optimizations)

5.1 Design of the Combined Kernel

In `conv_optimized.cpp`, we combine the best features of loop reordering, unrolling, register tiling, and AVX2 SIMD into a single high-performance kernel:

- **Register Tiling:** We keep 8 vector accumulators in AVX2 registers (covering $8 \times 8 = 64$ output pixels). Accumulation across the entire filter is done entirely in registers, and completed values are written to memory only once at the end.
- **Unrolling Across Vectors:** In pure SIMD, using a single accumulator causes a 4-cycle wait on each multiply-accumulate instruction. Unrolling across 8 independent vector accumulators gives the CPU independent instructions to execute in parallel, hiding instruction latency.
- **Kernel Weight Broadcasting:** Each filter weight is broadcast into an AVX2 register once and reused across all 8 vector accumulators (64 outputs).
- **Sequential Row Streaming:** We process the image in continuous 64-float horizontal chunks row-by-row. At any given time, only 3 input rows are active, which fit easily in L1-D cache while allowing the CPU stream prefetcher to run without interruption.

Omission of Spatial Tiling in `conv_optimized.cpp`: As demonstrated in Task 1B, 2D spatial cache tiling provided negligible speedup ($\sim 1.09\times$) on our hardware and actually caused slowdowns for narrow or square tile dimensions. Therefore, in `conv_optimized.cpp`, we intentionally omitted spatial cache tiling in favor of continuous row streaming, focusing optimization on 8-wide AVX2 SIMD, 8-way vector unrolling, and in-register accumulation.

5.2 Performance Synergy

When applied individually, each technique encounters a clear bottleneck:

- **Loop Reordering Alone ($1.15\times$):** Enables unit-stride memory access, but is limited by scalar instruction throughput.
- **Loop Unrolling Alone ($3.37\times$):** Reduces branch overhead and exposes parallelism, but still processes only one float per instruction.
- **Cache Tiling Alone ($1.09\times$):** Offers minimal benefit on its own because 3 rows already fit in L1 cache, while small 2D tiles interrupt hardware prefetching.
- **Pure SIMD Alone ($7.15\times$):** Computes 8 floats per instruction, but stalls on the 4-cycle latency of a single accumulator.

When combined, each technique removes the limitation of the others: SIMD provides high compute throughput, unrolling hides instruction latency, register tiling eliminates intermediate memory writes, and row streaming keeps active data in L1 cache. Together, they achieve an overall speedup of **$13.34\times$** (76.68 GFLOP/s) at $K = 3$ and **$19.67\times$** (104.85 GFLOP/s) at $K = 5$ (Table 5).

Table 5: Overall Performance and Speedup Summary ($H = 2048, W = 2048$, Averaged over 10 independent runs)

Optimization Stage	Kernel Size $K = 3$ (Graded)			Kernel Size $K = 5$ (Reference)		
	Mean Time	GFLOP/s	Speedup	Mean Time	GFLOP/s	Speedup
1. Naive Baseline	13.14 ms	5.75 GFLOP/s	1.00×	39.34 ms	5.33 GFLOP/s	1.00×
2. Loop Reorder	11.46 ms	6.59 GFLOP/s	1.15×	32.64 ms	6.42 GFLOP/s	1.21×
3. Loop Unroll (x8)	3.90 ms	19.38 GFLOP/s	3.37×	11.13 ms	18.84 GFLOP/s	3.53×
4. Cache Tiling	12.02 ms	6.28 GFLOP/s	1.09×	35.10 ms	5.97 GFLOP/s	1.12×
5. Pure SIMD (AVX2)	1.84 ms	41.08 GFLOP/s	7.15×	5.39 ms	38.91 GFLOP/s	7.30×
6. Fully Optimized	0.98 ms	76.68 GFLOP/s	13.34×	2.00 ms	104.85 GFLOP/s	19.67×

5.3 Evaluation Across Matrix and Kernel Sizes

Figures 6 and 7 compare the speedup of each optimization stage across varying matrix sizes and kernel dimensions.

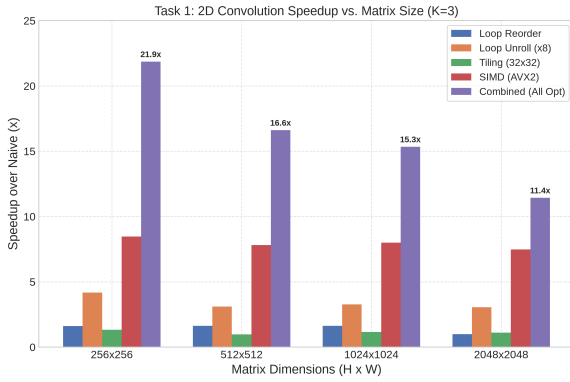


Figure 6: Speedup vs. Matrix Size ($K = 3$).

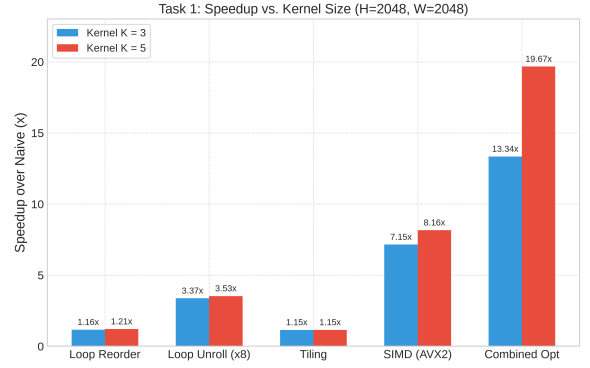


Figure 7: Speedup vs. Kernel Size ($H = 2048, W = 2048$).

Analysis of Performance Trends:

- Matrix Size Trends ($N = 256$ to 2048):** The highest speedup ($21.9\times$) is achieved on smaller matrices ($N \leq 512$), where the entire image fits in CPU cache and avoids DRAM traffic. For large matrices ($N = 2048$), memory bandwidth becomes more of a factor, but the combined kernel still sustains a strong $13.34\times$ speedup through continuous row streaming and register reuse.
- Kernel Size Trends ($K = 3$ vs. $K = 5$):** Increasing the kernel size from $K = 3$ to $K = 5$ raises the number of arithmetic operations per pixel from 9 to 25. Because input memory accesses are amortized over more computation, the processor achieves higher arithmetic intensity, boosting speedup from $13.34\times$ (76.68 GFLOP/s) to $19.67\times$ (104.85 GFLOP/s).